CSE 344 – System Programming
TERM PROJECT
Concurrent Stock Exchange Simulator
Due Date: June 5th – 23:59

Important Rules

• Implemented in C, must compile and run on at least on one Debian 12, Fedora 42, Centos 10 . Provide a Makefile for x86

• Identical or suspiciously similar submissions receive -100 points and are reported for academic misconduct.

• A report with uncut screenshots is mandatory. A submission without a report will not be evaluated.

• Late submissions are NOT accepted.

1. Overview

In this termproject you are expected to design and implement a concurrent stock exchange server and three corresponding client programs. The server should handle multiple simultaneous TCP clients using select() or poll() for I/O multiplexing (no thread-per-client). In addition to TCP, the server periodically broadcasts live stock price updates to UDP listener clients.

The project covers:

- TCP and UDP sockets used simultaneously
- All client connections managed in a single loop without threads
- Per-client state management: each connected TCP client has a username, type, and portfolio
- Dynamic price updates: BUY/SELL orders change stock prices, updated prices broadcast via UDP
- Partial read/write handling: TCP does not guarantee message boundaries
- Signal handling: server and clients exit cleanly on Ctrl+C and Ctrl+D

⚠ **The server is expected to be (must be) single-threaded. All TCP and UDP I/O must be handled inside one select() or poll() loop. Thread-per-client is strictly forbidden and will result in a major penalty.**

2. System Scenario

A stock exchange opens every morning. The server loads a list of stocks and their initial prices from a file. Three types of participants connect to the exchange:

- Trader: connects via TCP, submits BUY and SELL orders, checks their portfolio
- Analyst: connects via TCP, queries current stock prices and market reports
- Ticker: connects via UDP, receives periodic price broadcast updates

When a trader submits a BUY or SELL order, the server updates the stock price and immediately broadcasts the new price to all active ticker clients via UDP. Analysts can query current prices at any time. Each trader has a personal portfolio stored on the server: the server tracks how many shares each trader holds for each stock.

On SIGINT: the server sends a disconnect notice to all TCP clients, closes all file descriptors, and exits cleanly.

3. Programs to Implement

Four separate programs must be implemented and compiled:

Program	Protocol	Role
ServerTrd	TCP + UDP	Exchange center. Manages all clients, prices, portfolios.
trader	TCP	Submits BUY/SELL orders. Maintains a portfolio on the server.
analyst	TCP	Queries stock prices and market reports. Read-only.
ticker	UDP	Receives periodic price broadcast. No commands sent.

3.1 Command-Line Parameters

Flag	Parameter	Description
-p	<tcp_port>	TCP port for trader and analyst connections. Must be >= 1024.
-u	<udp_port>	UDP port for ticker broadcast. Must be >= 1024.
-s	<stocks.txt>	Path to stock list file. Must exist and be readable.
-l	<logfile>	Path to server log output file.

Missing or invalid parameters must print a usage message to stderr and exit with a non-zero code.

3.2 Server

./ServerTrd -p <tcp_port> -u <udp_port> -s <stocks.txt> -l <logfile>

- Loads stock list and initial prices from stocks.txt at startup
- Opens a TCP socket on tcp_port (accepts trader and analyst connections)
- Opens a UDP socket on udp_port and sends broadcast messages to 255.255.255.255 using setsockopt(SO_BROADCAST)
- Uses select() or poll() to multiplex all TCP client fds and the UDP socket in a single thread
- Maintains per-client state: username, type (TRADER/ANALYST), fd, line buffer, portfolio (TRADER only)

- Supports at least 10 simultaneous TCP clients
- Broadcasts updated stock prices via UDP after every BUY or SELL order
- Periodically broadcasts all stock prices via UDP every 5 seconds even without trades
- On SIGINT: sends SERVER SHUTDOWN to all TCP clients, closes all fds, exits cleanly

3.3 trader

./trader <server_ip> <tcp_port> <username>

- Connects to server via TCP
- Automatically sends JOIN TRADER <username> upon connection
- Reads commands from stdin, sends to server
- Prints all server responses to stdout
- When the user presses Ctrl+D, the client sends QUIT and exits cleanly.

3.4 analyst

./analyst <server_ip> <tcp_port> <username>

- Connects to server via TCP
- Automatically sends JOIN ANALYST <username> upon connection
- Can only send: PRICE, REPORT, LIST, QUIT
- Cannot send BUY or SELL: server rejects these with ERR UNAUTHORIZED
- When the user presses Ctrl+D, the client sends QUIT and exits cleanly.

3.5 ticker

./ticker <udp_port>

- Opens a UDP socket and binds to the given udp_port
- Listens for UDP broadcast messages from the server (SO_BROADCAST)
- Prints every received PRICE_UPDATE line to stdout
- Sends no messages to the server: receive only
- Exits on Ctrl+C

4. Input Files

4.1 stocks.txt

Each line defines one stock:

<symbol> <initial_price>

Example:

```
AAPL 150.00
GOOGL 2800.00
MSFT 380.00
TSLA 250.00
AMZN 3400.00
```

- symbol: uppercase alphanumeric, max 8 characters
- initial_price: positive float with 2 decimal places
- Lines with invalid format must be skipped silently

5. Protocol

All TCP messages are line-based: each message ends with \n. Maximum line length: 512 bytes. Partial reads must be handled with a per-client line buffer. Partial writes must also be handled correctly when sending responses.

5.1 Trader Commands (TCP)

Command	Description	Server Response
JOIN TRADER <username>	Register as trader. Must be first command.	OK JOIN <username> or ERR JOIN name_taken
BUY <symbol> <qty>	Buy qty shares of symbol.	OK BUY <symbol> <qty> <price> or ERR UNKNOWN_SYMBOL
SELL <symbol> <qty>	Sell qty shares. Must own enough shares.	OK SELL <symbol> <qty> <price> or ERR INSUFFICIENT_SHARES
PORTFOLIO	Request own portfolio.	OK PORTFOLIO <symbol>:<qty>,... or OK PORTFOLIO EMPTY
QUIT	Disconnect.	OK QUIT

5.2 Analyst Commands (TCP)

Command	Description	Server Response
JOIN ANALYST <username>	Register as analyst.	OK JOIN <username> or ERR JOIN name_taken
PRICE <symbol>	Query current price of symbol.	OK PRICE <symbol> <price> or ERR UNKNOWN_SYMBOL
REPORT	Get full market snapshot.	OK REPORT <symbol>:<price>,...
LIST	List all connected client usernames.	OK LIST <u1>,<u2>,...

QUIT	Disconnect.	OK QUIT

5.3 UDP Broadcast (Server to Ticker)

The server sends UDP broadcast messages to 255.255.255.255 on udp_port. Any ticker listening on that port receives them. No registration or connection is required from the ticker side. After every BUY or SELL, and every 5 seconds, the server broadcasts:

PRICE_UPDATE <symbol>:<price> <symbol>:<price> ...

Example:

PRICE_UPDATE AAPL:150.50 GOOGL:2800.00 MSFT:381.20 TSLA:248.70 AMZN:3400.00

Common error responses for both client types:

- ERR UNAUTHORIZED: command not allowed for this client type
- ERR UNKNOWN <command>: unrecognized command
- ERR TOOLONG: line exceeds 512 bytes
- ERR NOT_JOINED: client sent a command before JOIN. Server must reject all commands until JOIN is received.
- SERVER SHUTDOWN: server is closing, client must disconnect

5.4 Price Update Mechanism

Stock prices change dynamically based on trader orders:

- BUY <symbol> <qty>: price increases by qty * 0.01
- SELL <symbol> <qty>: price decreases by qty * 0.01
- Price must never go below 0.01
- Price is stored and reported with 2 decimal places

After every price change, the server immediately sends a UDP broadcast with all current prices. The server also sends a periodic broadcast every 5 seconds regardless of trading activity.

> ⚠ **The 5-second broadcast must occur regardless of client activity. Track the last broadcast time using clock_gettime() or time() and recalculate the remaining timeout before each select() call.**

5.5 Per-Client State

The server maintains a state record for each connected TCP client:

- username: registered name, from JOIN command
- type: TRADER or ANALYST
- fd: file descriptor

- line_buf: partial read buffer, handles incomplete TCP messages
- portfolio: TRADER only, symbol to quantity mapping

The server must maintain a client_t struct for each connected client. A minimum required structure is:

```
typedef struct {
    int      fd;
    char     username[32];
    char     type[16];       /* TRADER or ANALYST */
    char     line_buf[512];  /* partial read buffer */
    /* portfolio: design is left to you (TRADER only) */
} client_t;
```

When a client disconnects (QUIT or hangup), its state must be freed and its fd removed from the fd_set. Portfolio data is not persisted: it is lost on disconnect.

> ⚠ **Two clients with the same username must be rejected with ERR JOIN name_taken regardless of type. An analyst and a trader cannot share the same username.**

6. Outputs

The server writes all log lines to both stdout and the log file specified by -l simultaneously. This is mandatory. Clients write only to stdout.

6.1 Output Keywords

All output lines follow the format: [SOURCE] KEYWORD fields. The table below covers both server and client output lines.

Keyword	Source	Required Fields
SERVER_START	[SERVER]	tcp_port=N udp_port=M stocks=K
CLIENT_CONNECTED	[SERVER]	fd=N ip=X.X.X.X
JOIN	[SERVER]	username=X type=TRADER/ANALYST fd=N
BUY	[SERVER]	trader=X symbol=Y qty=N old_price=P new_price=Q
SELL	[SERVER]	trader=X symbol=Y qty=N old_price=P new_price=Q
PRICE_BROADCAST	[SERVER]	trigger=TRADE/PERIODIC
CLIENT_DISCONNECTED	[SERVER]	username=X reason=QUIT/hangup
SHUTDOWN	[SERVER]	signal=SIGINT
CLEANUP_DONE	[SERVER]	clients=0
CONNECTED	[TRADER/ANALYST username]	server=IP:port

SENT	[TRADER/ANALYST username]	command text
RECEIVED	[TRADER/ANALYST username]	server response text
DISCONNECTED	[TRADER/ANALYST username]	reason=QUIT/shutdown
LISTENING	[TICKER]	udp_port=N
RECEIVED	[TICKER]	PRICE_UPDATE symbol:price ...

⚠ **Output line prefix format is mandatory. [SERVER], [TRADER username], [ANALYST username], [TICKER]. Wrong prefix or missing fields will fail grading checks.**

6.2 Example Server Log (stdout + server.log)

```
[SERVER] SERVER_START tcp_port=8080 udp_port=8081 stocks=5
[SERVER] CLIENT_CONNECTED fd=4 ip=127.0.0.1
[SERVER] JOIN username=alice type=TRADER fd=4
[SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[SERVER] JOIN username=carol type=ANALYST fd=5
[SERVER] BUY trader=alice symbol=AAPL qty=100 old_price=150.00 new_price=151.00
[SERVER] PRICE_BROADCAST trigger=TRADE
[SERVER] CLIENT_DISCONNECTED username=alice reason=QUIT
[SERVER] SHUTDOWN signal=SIGINT
[SERVER] CLEANUP_DONE clients=0
```

6.3 Example Client Output (stdout only)

Trader:

```
[TRADER alice] CONNECTED server=127.0.0.1:8080
[TRADER alice] SENT JOIN
[TRADER alice] SENT BUY AAPL 100
[TRADER alice] RECEIVED OK BUY AAPL 100 151.00
[TRADER alice] SENT PORTFOLIO
[TRADER alice] RECEIVED OK PORTFOLIO AAPL:100
[TRADER alice] DISCONNECTED reason=QUIT
```

Analyst:

```
[ANALYST carol] CONNECTED server=127.0.0.1:8080
[ANALYST carol] SENT PRICE AAPL
[ANALYST carol] RECEIVED OK PRICE AAPL 151.00
[ANALYST carol] DISCONNECTED reason=shutdown
```

Ticker:

```
[TICKER] LISTENING udp_port=8081
[TICKER] RECEIVED PRICE_UPDATE AAPL:151.00 GOOGL:2800.00 MSFT:380.00 TSLA:250.00 AMZN:3400.00
[TICKER] RECEIVED PRICE_UPDATE AAPL:151.00 GOOGL:2800.00 MSFT:380.00 TSLA:250.00 AMZN:3400.00
```

7. Signal Handling and Shutdown

7.1 Server Shutdown (Ctrl+C)

On SIGINT (Ctrl+C) sent to the server:

- The signal handler sets a volatile sig_atomic_t flag using only async-signal-safe operations.
- The main select() loop detects the flag and enters shutdown mode.
- SERVER SHUTDOWN is sent to all connected TCP clients.
- All TCP and UDP file descriptors are closed.
- SHUTDOWN and CLEANUP_DONE are logged.
- No zombie processes or orphan file descriptors may remain after exit.

7.2 Client Exit (Ctrl+D)

When the user presses Ctrl+D on a trader or analyst terminal:

- The client sends QUIT to the server.
- The server responds with OK QUIT and logs CLIENT_DISCONNECTED reason=QUIT.
- The client closes the connection and exits.
- Ticker clients exit on Ctrl+C only, as they send no commands.

8. Submission

Compress as StudentID_Final.zip. Do not include binaries or object (*.o) files.

```
StudentID/
├── Makefile
├── ServerTrd.c
├── trader.c
├── analyst.c
├── ticker.c
├── (and any additional .c / .h files)
└── StudentID_report.pdf
```

Makefile must compile all four programs with gcc -Wall -Wextra -std=c11 and include make clean.

Example:

```
make       # builds server, trader, analyst, ticker
make clean  # removes all binaries and .o files
```

9. Mandatory Test Scenarios

Run all scenarios and include uncut terminal screenshots in your report. Each scenario requires multiple terminals open simultaneously. Screenshot each terminal separately.

Base server command:

./ServerTrd -p 8080 -u 8081 -s stocks.txt -l server.log

All tests use the same stocks.txt file (30 stocks).

#	Scenario	What to Verify
1	2 traders (alice, bob) connect. alice sends BUY AAPL 100. bob sends BUY AAPL 50. Both press Ctrl+D when done.	Server log shows two sequential BUY entries with correct price updates. alice PORTFOLIO shows AAPL:100, bob PORTFOLIO shows AAPL:50. Final price = initial + 1.50.
2	3 traders + 2 analysts + 2 tickers connect. Traders send BUY and SELL on different stocks. Analyst1 sends LIST, analyst2 sends REPORT. All TCP clients press Ctrl+D when done.	LIST contains all connected usernames. REPORT contains all 30 stocks with updated prices. All tickers receive PRICE_UPDATE after every trade. Server log shows all events correctly.
3	1 trader + 1 analyst connect. Trader sends BUY AAPL 100. Trader then sends SELL AAPL 999. Analyst sends PRICE AAPL. Both press Ctrl+D.	SELL returns ERR INSUFFICIENT_SHARES. No UDP broadcast after failed SELL. Analyst PRICE shows price updated only by successful BUY. Server continues serving normally.
4	1 trader + 1 analyst + 1 ticker connect. Trader sends BUY on 3 different stocks . Analyst sends REPORT then LIST. Trader and Analyst presses Ctrl+D.	REPORT includes all 30 stocks with 3 updated prices. LIST shows both connected usernames. Ticker received PRICE_UPDATE 3 times. Partial read handling verified: REPORT is a long message.
5	3 traders connect. Trader1 process is killed with kill -9. Immediately a new trader connects with the same username as Trader1. Trader2 and Trader3 press Ctrl+D when done.	Server detects hangup and logs reason=hangup. New trader with same username is accepted (state cleaned up). Trader2 and Trader3 are unaffected throughout.
6	2 traders + 1 ticker active. Trader sends BUY. Send SIGINT to server (Ctrl+C). Immediately restart server on same ports. New trader connects and sends BUY. Run the server under valgrind during this scenario.	On SIGINT: all TCP clients receive SERVER SHUTDOWN. CLEANUP_DONE clients=0 logged. Server restarts immediately (SO_REUSEADDR works). New trader connects successfully. Valgrind: 0 leaks, 0 errors. No zombie processes (verify with ps).

⚠ **All screenshots must be uncut, showing the full terminal from command prompt through program exit. Cropped or edited screenshots will not be accepted.**

10. Report Requirements

The report must explain design decisions, not just describe the code. Diagrams are required.

Section	Content
Title Page	Name, ID, course
I/O Multiplexing Design	How the server manages all TCP clients and the UDP socket in a single loop. How the 5-second periodic broadcast timeout is implemented without threads or sleep().
Per-Client State	Structure of client_t. How portfolio is managed. How state is cleaned up on disconnect.
Price Update Mechanism	How BUY/SELL changes price. How UDP broadcast is triggered. Formula used.
Partial Read/Write Handling	How line buffers handle incomplete TCP messages.
Signal Handling and Shutdown	Step-by-step: server Ctrl+C sequence and client Ctrl+D sequence.
Test Scenarios	Uncut screenshots for all 6 mandatory tests with explanation.
Challenges	At least 2 concrete problems encountered and how they were resolved.

11. Grading and Penalties

Grading is holistic. A missing or non-functional component affects the correctness of all criteria that depend on it. If select() is not used, multi-client handling cannot be graded. If per-client state is absent, portfolio and price update correctness cannot be verified.

Grading

Criterion	Points
TCP server: select/poll loop, multi-client, correct lifecycle	15
UDP: periodic broadcast + trade-triggered broadcast, ticker receives correctly	10
Protocol correctness: all commands, correct responses, ERR cases	10
Per-client state: portfolio tracking, type enforcement, cleanup on disconnect	15
Price update mechanism: correct formula, broadcast after every trade	10
SIGINT: all clients notified, all fds closed, CLEANUP_DONE logged	5
Log format correctness (server + clients)	10
Compilation, Makefile (all 4 programs), zero -Wall -Wextra warnings	5
Report: all sections, diagrams, 6 test screenshots uncut	20
TOTAL	100

Penalties

Condition	Outcome
Code does not compile	-100
No report submitted	Not evaluated
No Makefile	Not evaluated
Late submission	Not accepted
Report submitted but screenshots missing or cropped	-80
Missing or broken Makefile	-80
Thread-per-client used instead of select/poll	-60
Server handles only one TCP client at a time	-60
UDP broadcast absent, ticker receives nothing	-40
Periodic broadcast implemented with sleep() or a thread	-40
Per-client state absent, no portfolio tracking	-40
Analyst can send BUY/SELL without rejection	-30
Partial read/write not handled	-30
Duplicate username accepted	-25
Price formula incorrect	-25
Crash on invalid or unknown command	-25
File descriptor leak after client disconnect	-25
Each -Wall warning (first 10)	-2 each
More than 10 -Wall warnings	-25